

Approches classiques de l'IA

Chapitre 3 : Résolution de problèmes par exploration

Hadrien Mélot
Service d'Algorithmique
Département d'Informatique
Faculté des Sciences



Chapitre 3. Résolution de problèmes par exploration

- 1 Agents “Problem-solving”
- 2 Formulation de problèmes
- 3 Exemples de problèmes
- 4 Stratégies d’exploration de base (non informées)
- 5 Stratégies d’exploration informées (heuristiques)
- 6 Résumé du chapitre

Agents "Problem-solving"

Agents "Problem-solving"

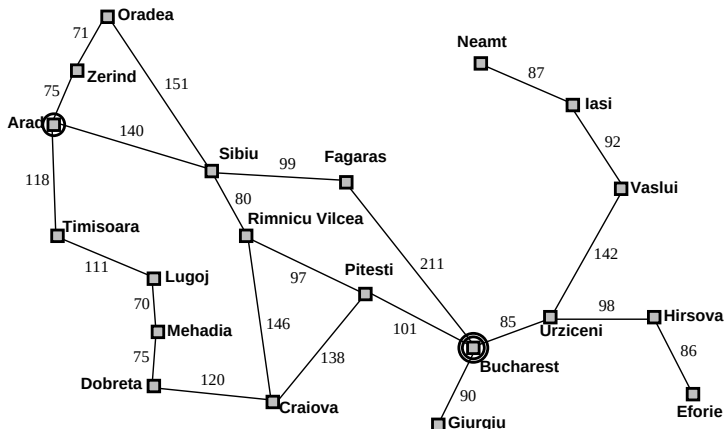
Dans ce chapitre, on décrit un type d'agent basé sur des **objectifs** (cf. chapitre 2) destinés à résoudre un certain type de problèmes : **problem-solving agents**.

Sous certaines hypothèses, l'idée sera de modéliser le problème sous la forme d'un graphe où chaque **état** du monde est représenté par un nœud et où les actions possibles de l'agent pour passer d'un état à l'autre sont représentées par des arcs. L'**objectif** est d'arriver dans un état qui permet d'atteindre son but.

Exemple : trouver sa route d'une ville A vers une ville B , via une séquence de villes traversées.

Exemple : Roumanie

Un agent se trouve en Roumanie, dans la ville d'Arad. Il doit se rendre à Bucarest.



Types d'environnements

Pour permettre la modélisation de problèmes divers sous la forme d'un parcours de graphe, il faut respecter certaines **hypothèses**.

On va supposer que l'environnement est

- **discret** (à chaque état, il y a un nombre fini d'actions possibles) ;
- **complètement observable** (l'agent sait quel état est atteint par quelle action) ;
- **déterministe** (chaque action n'a qu'une seule conséquence, i.e., suivre un arc nous amène dans un seul état possible).

Exemple : C'est le cas du problème de l'agent qui doit se déplacer en Roumanie. Le graphe nous est donné via la carte.

Agents "Problem-solving"

On suppose donc que l'environnement est complètement observable et déterministe.

Forme restreinte d'agent : "formuler - explorer - exécuter"

- **formulation** : spécification d'un **objectif** (goal) et d'un **problème** ;
- **exploration** : recherche d'une séquence d'actions qui résout le problème ;
- **exécution** : exécution de la séquence d'action les "yeux fermés".

⇒ **les perceptions sont ignorées**. On parle d'exécution **offline**. On sait à l'avance ce qu'on va faire. L'environnement le permet : connaissance complète.

Agents "Problem-solving"

function SIMPLE-PROBLEM-SOLVING-AGENT(*percept*) **returns** an action

persistent: *seq*, an action sequence, initially empty
state, some description of the current world state
goal, a goal, initially null
problem, a problem formulation

state ← UPDATE-STATE(*state*, *percept*)

if *seq* is empty **then**

goal ← FORMULATE-GOAL(*state*)

problem ← FORMULATE-PROBLEM(*state*, *goal*)

seq ← SEARCH(*problem*)

action ← RECOMMENDATION(*seq*, *state*)

seq ← REMAINDER(*seq*, *state*)

return *action*

Exemple : Roumanie

En vacances en Roumanie ; actuellement à Arad. L'avion décole le lendemain de Bucarest.

Formulation de l'objectif (FORMULATE-GOAL) :

- Etre à Bucarest

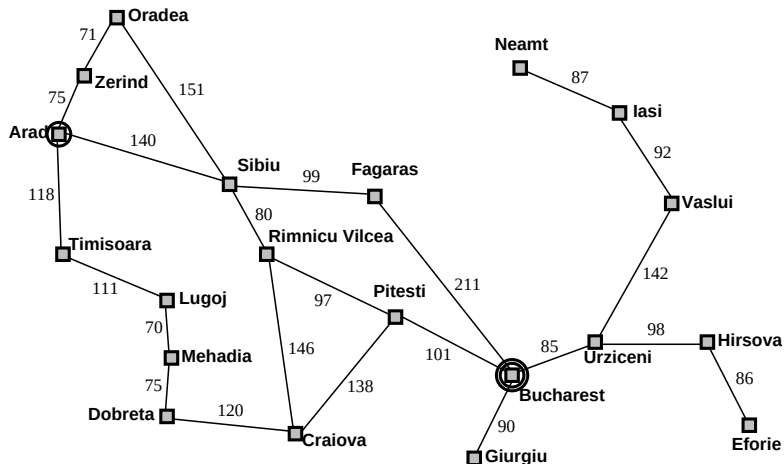
Formulation du problème (FORMULATE-PROBLEM) :

- Etats : diverses villes
- Actions : conduire entre les villes

Trouver une solution (SEARCH) :

- Retourne une sequence de villes, e.g., Arad, Sibiu, Fagaras, Bucarest

Exemple : Roumanie



Formulation de problèmes

Formulation d'un problème à état unique

Définition

Un **problème** est défini par cinq composantes :

- Un **état initial** (INITIAL-STATE).
- Les **actions** possibles spécifiées par une fonction $ACTIONS(s)$ qui retourne l'ensemble des actions qui peuvent être exécutées à partir de l'état s .
- Un **modèle de transition** spécifié par une fonction $RESULT(s, a)$ qui retourne l'état (unique) obtenu en réalisant l'action a à partir de l'état s .
- Un **test objectif** ($GOAL-TEST(s)$) qui détermine si un état donné s est un état dans lequel l'objectif est atteint.
- Une **fonction de coût** (PATH-COST) qui assigne un coût numérique à une séquence d'actions. Elle est définie par la somme des coûts $c(s, a, s')$, où $c(s, a, s')$ représente le coût de l'action a permettant de passer de l'état s à l'état s' (**step cost**).

Formulation d'un problème à état unique

Exemple

- Etat initial : $In(Arad)$.
- Depuis l'état $In(Arad)$, l'ensemble des actions applicables sont

$$\{Go(Sibiu), Go(Timisoara), Go(Zerind)\}.$$

- $RESULT(In(Arad), Go(Zerind)) = In(Zerind)$.
- Un test objectif peut être
 - ▶ explicite, e.g., $s = In(Bucharest)$
 - ▶ implicite, e.g., $EchecEtMat(s)$
- fonction de coût : somme des distances, nombre d'actions exécutées, etc.

Formulation d'un problème à état unique

Définition

L'**espace d'états** (state space) d'un problème est l'ensemble de tous les états atteignables depuis l'état initial en suivant n'importe quelle séquence d'actions applicable.

L'espace des états d'un problème est défini implicitement par

- l'état initial ;
- les actions possibles ;
- le modèle de transition.

Remarque : Problème à "état unique"

- $\neq$ un seul état (beaucoup d'états possibles dans un espace d'états)
- $= \text{RESULT}(s, a)$ retourne un seul état possible

Formulation d'un problème à état unique

Un espace d'états définit un **graphe orienté** (pondéré) dont les sommets sont les états et les arcs sont les actions. On appelle ce graphe le **graphe d'états**. Le poids d'un arc allant de s à s' via l'action a est $c(s, a, s')$.

D'un point de vue implémentation, on accède à $c(s, a, s')$ via *problem*.STEP-COST(s, a) (s' est implicite car connu via *problem*.RESULT(s, a))

Par ex. : la carte de la Roumanie où les liaisons entre les villes représentent deux actions : rouler dans un sens ou dans l'autre. Les poids sont les distances en *km*.

Formulation d'un problème à état unique

Définition

Un **chemin** (path) dans un espace d'états est une séquence d'états connectés par une séquence d'actions.

Un chemin dans un espace d'états correspond donc exactement avec la notion de chemin dans le graphe associé.

⇒ fonction de coût = somme des coûts (poids) des arcs d'un chemin.

Formulation d'un problème à état unique

Définition

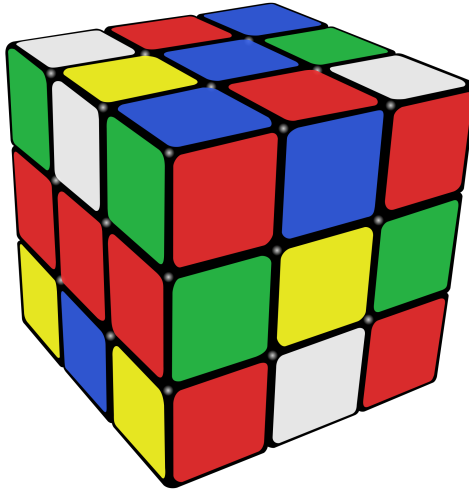
Une **solution** est une séquence d'actions menant de l'état initial à un état objectif. Son **coût** est la valeur de sa séquence d'actions évaluée par la fonction de coût.

Définition

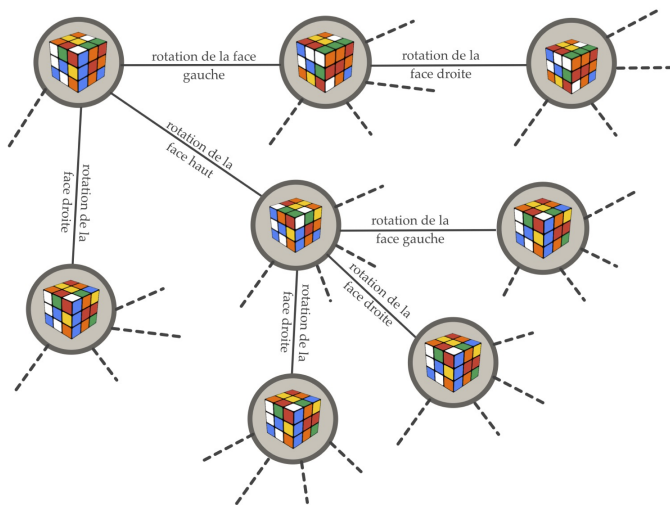
Une **solution optimale** est une solution de coût minimum parmi toutes les solutions.

Exemples de problèmes

Exemple : Rubik's cube

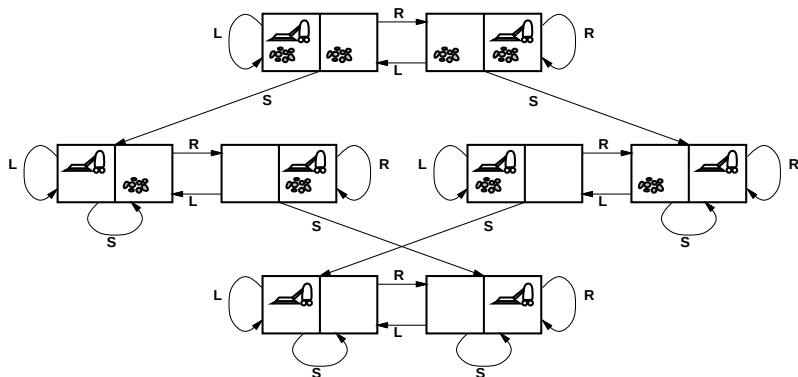


Exemple : Rubik's cube

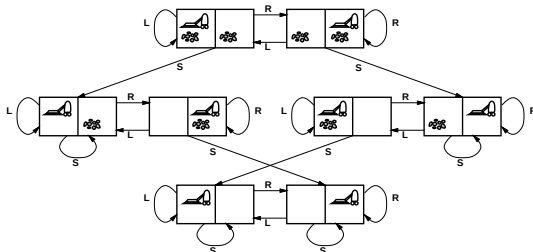


Nombre d'états = 43.252.003.274.489.856.000

Exemple : robot aspirateur



Exemple : robot aspirateur



Etats ?? : Localisation du robot et propreté des 2 carrés (entiers, ignore la quantité de poussière etc.). Huit états possibles ¹

Etat initial ?? : Tout état peut être choisi comme état initial

Actions ?? : Chaque état a 3 actions (*Gauche*, *Droite*, *Aspire*)

Modèle de transition ?? : Voir graphe

Test objectif ?? : Tous les carrés sont propres

Fonction de coût ?? : 1 par action (= longueur de la séquence)

1. Avec n carrés : $n \times 2^n$ états.

Exemple : n -puzzle



Remarque : n -puzzle = puzzle avec n pièces (ici, 8)

Exemple : Casse-tête “8-puzzle”

7	2	4
5		6
8	3	1

Start State

1	2	3
4	5	6
7	8	

Goal State

Exemple : Casse-tête “8-puzzle”

7	2	4
5		6
8	3	1

Start State

1	2	3
4	5	6
7	8	

Goal State

Etats ?? : localisations des pièces (entiers, on ignore les positions intermédiaires) = $9!/2 = 181440$ états atteignables²

Actions ?? : bouger le “blanc” à gauche, droite, haut ou bas

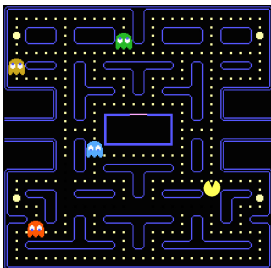
Modèle de transition ?? : Défini par les mouvements possibles. Ex. : si on applique *Gauche* à l’état initial, on se retrouve dans un état où le blanc et 5 ont été permutés et inversément. Il y a donc deux arcs entre ces 2 états.

Test objectif ?? : = état objectif (donné explicitement)

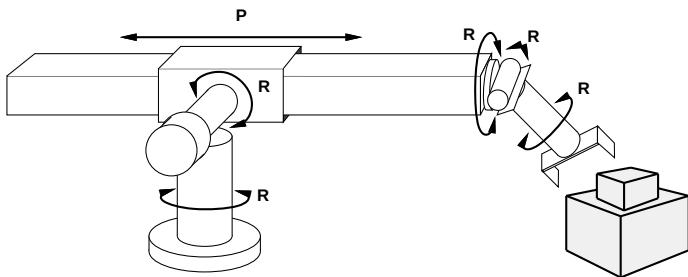
Fonction de coût ?? : 1 par mouvement

2. La famille des n -Puzzles = problèmes NP-difficiles (cf. cours “Calculabilité et complexité”).

Exemple : pathfinding dans les jeux



Exemple “réel” : assemblage robotique



Etats ?? : coordonnées des angles des actionneurs (réels), partie de l'objet à assembler

Actions ?? : mouvements (continus) des actionneurs

Test objectif ?? : assemblage complet

Fonction de coût ?? : temps d'exécution

Quelques autres problèmes réels

- problèmes de routing (GPS, mappy, routing de stream vidéos, systèmes de “travel-planning” aérien, etc.) : plus court chemin, coût minimum
- problèmes de tournées (problème du voyageur de commerce, tournées postales, etc.) : circuit le plus court
- layout VLSI : intégration à très grande échelle (Very Large Scale Integration) : technologie de circuit intégré (CI) dont la densité d'intégration permet de supporter plus de 100 000 composants électroniques sur une même puce : minimiser la surface d'une puce.
- beaucoup d'autres problèmes d'optimisation (recherche opérationnelle, théorie des graphes, etc.).

Stratégies d'exploration de base (non informées)

Algorithmes d'exploration arborescente

Idée de base :

- offline, simule l'exploration de l'espace d'états
- génère les successeurs des états déjà explorés (expansion d'états ajoutés dans la **frontière**)
- quel état choisir pour réaliser l'expansion suivante ?
⇒ **stratégie d'exploration**
- s'arrête quand un état choisi est un état objectif.

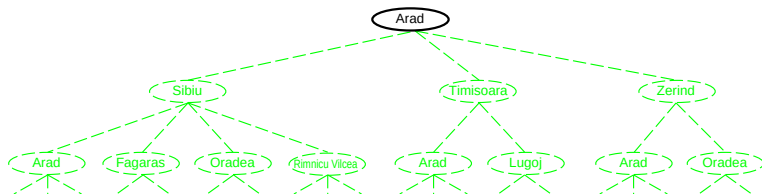
Algorithmes d'exploration arborescente

De manière très informelle :

```
function TREE-SEARCH(problem) returns a solution, or failure
  initialize the frontier using the initial state of problem
  loop do
    if the frontier is empty then
      return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then
      return the corresponding solution
    else
      expand chosen node and add resulting nodes to the frontier
  end
```

Exemple d'exploration arborescente

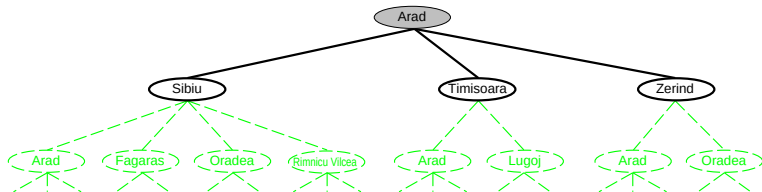
Etat initial : Arad.



En blanc : la frontière.

Exemple d'exploration arborescente

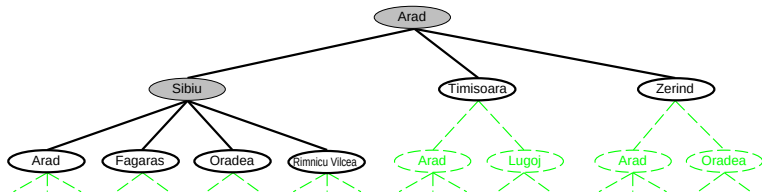
Après l'expansion d'Arad.



En blanc : la frontière. En gris : déjà exploré.

Exemple d'exploration arborescente

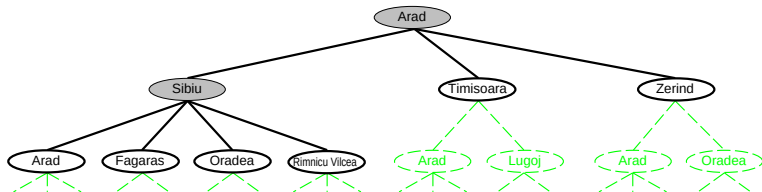
Après l'expansion de Sibiu.



En blanc : la frontière. En gris : déjà exploré.

Exemple d'exploration arborescente

Après l'expansion de Sibiu.



Apparition de boucles ! (Nous sommes de retour à Arad)

Exemple d'exploration arborescente

Les boucles sont-elles problématiques ?

- elles définissent une exploration arborescente **infinie** des états (même si eux sont en # fini, par ex., dans le cas de la carte de la Roumanie : 20 états) ;
- elles peuvent faire échouer certains algorithmes ;
- **mais** comme les coûts sont non négatifs, une boucle ne mènera pas à une bonne solution et a fortiori à une solution optimale : on peut les rejeter en mémorisant les noeuds déjà explorés (GRAPH-SEARCH)

Mémoriser les nœuds visités

En *italique* : ce qui est ajouté par rapport à TREE-SEARCH.

```

function GRAPH-SEARCH(problem) returns a solution, or failure
  initialize the frontier using the initial state of problem
  initialize the explored set to be empty
  loop do
    if the frontier is empty then
      return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then
      return the corresponding solution
    else
      add the node to the explored set
      expand chosen node and add resulting nodes to the ...
        frontier only if not in the frontier or the explored set
  end
  
```

TREE-SEARCH (TS) vs GRAPH-SEARCH (GS)

Une exploration GRAPH-SEARCH (GS)

- évite les boucles
- conserve l'ensemble des états explorés, en plus de la frontière (ce qui a un coût en mémoire)
- **mais** il y a au plus **une** copie de chaque état en mémoire (dans “frontière” ou “explorés”)
- effectue une exploration systématique des états, un par un

Exemple : pour le problème du chemin en Roumanie, TS peut boucler indéfiniment (et faire exploser la mémoire) alors que GS non et aura au maximum 20 nœuds en mémoire.

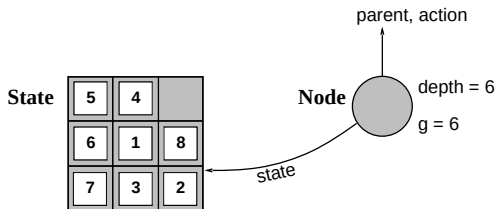
Implémentation : états vs noeuds

- Un **état** est une (représentation d'une) configuration physique
- Un **noeud** est une structure de données (objet) faisant partie d'un arbre d'exploration.

Un noeud n contient les attributs suivants :

- $n.STATE$: l'état dans l'espace d'états auquel ce noeud correspond ;
- $n.PARENT$: le noeud dans l'arbre d'exploration qui a généré ce noeud ;
- $n.ACTION$: l'action qui a été appliquée au parent pour générer ce noeud ;
- $n.PATH-COST$: le coût – en général noté $g(n)$ – du chemin de l'état initial vers ce noeud.

Implémentation : états vs noeuds



- Dans l'exemple du casse-tête, le coût $g(n)$ est équivalent à la profondeur dans l'arbre
- A partir d'un noeud parent, il est facile de voir comment on peut calculer les attributs d'un noeud enfant (voir CHILD-NODE ci-après)
- Il est également facile d'imaginer comment retrouver une solution à partir d'un noeud correspondant à un état objectif via l'attribut PARENT (fonction récursive SOLUTION laissée en exercice)

Implémentation : créer un nouveau noeud

```

function CHILD-NODE(problem, parent, action) returns a node
  n ← MAKE-NODE(problem.RESULT(parent.STATE, action))
  n.PARENT ← parent
  n.ACTION ← action
  n.PATH-COST ← parent.PATH-COST
                  + problem.STEP-COST(parent.STATE, action)

  return n

```

```

function MAKE-NODE(state) returns a node
  n ← a new NODE
  n.STATE ← state
  return n

```

Remarque : MAKE-NODE sera utile également pour créer le noeud initial. Dans ce pseudo-code, on omet de mettre les attributs non définis à *null*³ (implicite). En pratique, il faut y penser si nécessaire !

3. None en Python ou null en Java.

Implémentation : stocker des noeuds

On va utiliser des files/piles pour stocker les noeuds (par ex. la frontière) :

- IS-EMPTY(*queue*) : retourne vrai ssi il n'y a plus d'éléments dans la file ;
- POP(*queue*) : retire le premier élément de la file et retourne cet élément ;
- INSERT(*queue*, *element*) : retourne la file résultant de l'insertion d'un élément ;
- EXTEND(*q1*, *q2*) : retourne la file résultant de l'insertion de tous les éléments de *q2* dans *q1*.

L'ordre dans lequel un élément sera inséré (INSERT et EXTEND) va déterminer le type de la file utilisée : cela permettra différentes stratégies d'exploration (voir ci-après).

Implementation : expansion

```
function EXPAND(node, problem) returns a set of nodes  
  successors ← an empty queue (order defined by INSERT and EXTEND)  
  for each action in problem.ACTIONS(node.STATE) do  
    n ← CHILD-NODE(problem, node, action)  
    INSERT(n, successors)  
  return successors
```

Implémentation : exploration arborescente générale

```
function TREE-SEARCH(problem) returns a solution, or failure
  frontier ← INSERT(frontier, MAKE-NODE(problem.INITIAL-STATE))
  loop do
    if IS-EMPTY(frontier) then return failure
    node ← POP(frontier)
    if problem.GOAL-TEST(node.STATE) then return node
    frontier ← EXTEND(frontier, EXPAND(node, problem))
```

Implémentation : types de file pour les noeuds

L'ordre dans lequel un élément sera inséré va déterminer le type de la file utilisée : FIFO, LIFO ou priority queue.

- **file** (FIFO : first in, first out — queue) : insertion à la fin
- **pile** (LIFO : last in, first out — stack) : insertion au début
- **priority queue** : insertion des éléments en fonction d'un ordre de priorité

Implémentation : types de file pour les noeuds

Exemple : (a) on insère (dans l'ordre) 1, 2, puis 3 ; (b) on retire un élément (POP) ; (c) on insère 4 ; (d) on retire un élément.

File FIFO (insertion à la fin, priorité au premier arrivé)

(a) : $queue = 1 \rightarrow 2 \rightarrow 3$

(b) : $queue = 2 \rightarrow 3$; retourne 1

(c) : $queue = 2 \rightarrow 3 \rightarrow 4$

(d) : $queue = 3 \rightarrow 4$; retourne 2

$x \rightarrow y$ signifie “le suivant de x est y ”

Implémentation : types de file pour les noeuds

Exemple : (a) on insère (dans l'ordre) 1, 2, puis 3 ; (b) on retire un élément (POP) ; (c) on insère 4 ; (d) on retire un élément.

Pile LIFO (insertion au début, priorité au dernier arrivé)

(a) : $stack = 3 \rightarrow 2 \rightarrow 1$
 (b) : $stack = 2 \rightarrow 1$; retourne 3
 (c) : $stack = 4 \rightarrow 2 \rightarrow 1$
 (d) : $stack = 2 \rightarrow 1$; retourne 4

$x \rightarrow y$ signifie “le suivant de x est y ”

Implémentation : types de file pour les noeuds

Exemple : (a) on insère (dans l'ordre) 1, 2, puis 3 ; (b) on retire un élément (POP) ; (c) on insère 4 ; (d) on retire un élément.

Priority queue (priorité libre, par ex. priorité donnée aux nombres impairs, puis à la plus grande valeur)

(a) : $stack = 3 \rightarrow 1 \rightarrow 2$
 (b) : $stack = 1 \rightarrow 2$; retourne 3
 (c) : $stack = 1 \rightarrow 4 \rightarrow 2$
 (d) : $stack = 4 \rightarrow 2$; retourne 1

$x \rightarrow y$ signifie “le suivant de x est y ”

Stratégies d'exploration

Une stratégie est définie par l'**ordre** dans lequel les noeuds (de la frontière) seront explorés.

Une stratégie est évaluée selon quatre dimensions :

- **complétude** — trouve-t-elle toujours une solution s'il en existe ?
- **optimalité** — trouve-t-elle toujours la solution optimale ?
- **complexité en temps** — nombre de noeuds générés lors de l'exploration⁴
- **complexité en espace** — nombre maximum de noeuds en mémoire

4. En supposant que le travail pour un noeud est en $O(1)$.

Stratégies d'exploration

Les complexités en temps et en espace (dans le pire des cas) sont mesurées en termes de

- b — (**branching**) le facteur de branchement (nombre d'enfants) maximum dans un arbre d'exploration
- d — (**depth**) la profondeur minimum d'une solution optimale (distance minimum de l'état initial à un état objectif)
- m — (**maximum depth**) la profondeur maximum de l'espace d'états (peut être ∞)

Stratégies d'exploration non-informées

Uninformed search strategies / Blind search

Les stratégies **non-informées** utilisent uniquement l'information disponible dans la définition du problème. Exemples :

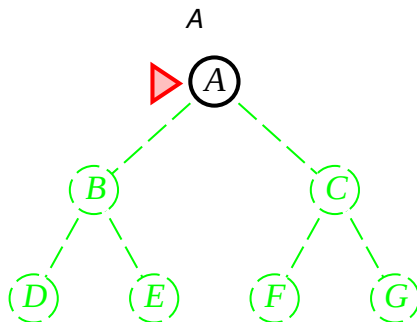
- **Breadth-first search** (BFS, exploration en largeur d'abord)
- **Uniform-cost search** (exploration à coût uniforme)
- **Depth-first search** (DFS, exploration en profondeur d'abord)
- **Depth-limited search** (exploration en profondeur limitée)
- **Iterative deepening search** (exploration avec profondeur modifiée itérativement)

Breadth-first search

Expansion : d'abord les noeuds les moins profonds.

Implémentation : GRAPH-SEARCH tel que

- test objectif appliqué quand noeud **généré** (plutôt que quand sélectionné pour expansion)
- *frontier* est une file FIFO :

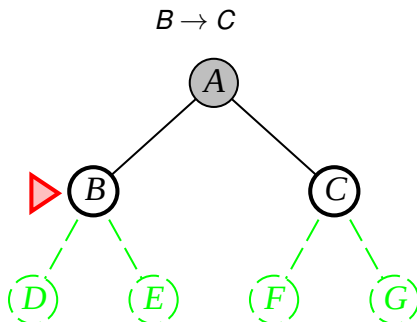


Breadth-first search

Expansion : d'abord les noeuds les moins profonds.

Implémentation : GRAPH-SEARCH tel que

- test objectif appliqué quand noeud **généré** (plutôt que quand sélectionné pour expansion)
- *frontier* est une file FIFO :

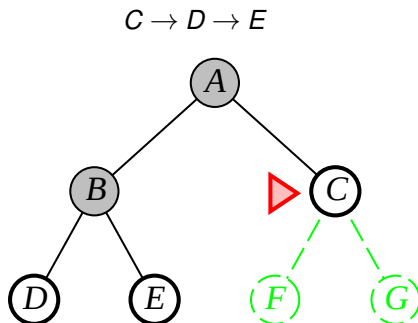


Breadth-first search

Expansion : d'abord les noeuds les moins profonds.

Implémentation : GRAPH-SEARCH tel que

- test objectif appliqué quand noeud **généré** (plutôt que quand sélectionné pour expansion)
- *frontier* est une file FIFO :

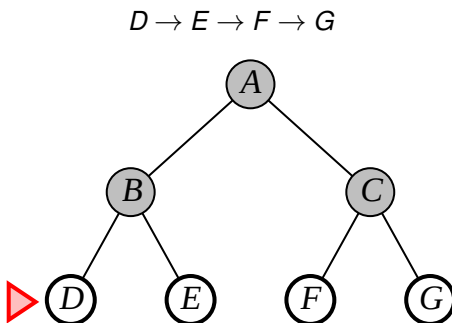


Breadth-first search

Expansion : d'abord les noeuds les moins profonds.

Implémentation : GRAPH-SEARCH tel que

- test objectif appliqué quand noeud **généré** (plutôt que quand sélectionné pour expansion)
- *frontier* est une file FIFO :



Breadth-first search

```

function BREADTH-FIRST-SEARCH(problem) returns sol or failure
  node ← MAKE-NODE(problem.INITIAL-STATE)
  node.PATH-COST ← 0
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  frontier ← a FIFO queue with node as the only element
  explored ← an empty set
  loop do
    if IS-EMPTY(frontier) then return failure
    node ← POP(frontier)
    add node.STATE to explored
    for each action in problem.ACTIONS(node.STATE) do
      child ← CHILD-NODE(problem, node, action)
      if child.STATE is not in explored or frontier then
        if problem.GOAL-TEST(child.STATE) then
          return SOLUTION(child)
        frontier ← INSERT(child, frontier)

```


Propriétés du breadth-first search

Complet ??

5. C'est le cas si tous les coûts sont constants (relativement fréquent, cf. robot aspirateur).

Propriétés du breadth-first search

Complet ?? Oui (si b est fini)

Optimal ??

5. C'est le cas si tous les coûts sont constants (relativement fréquent, cf. robot aspirateur).

Propriétés du breadth-first search

Complet ?? Oui (si b est fini)

Optimal ?? Oui si la fonction de coût est une fonction non décroissante en la profondeur du noeud⁵ ; pas optimal en général

Temps ??

5. C'est le cas si tous les coûts sont constants (relativement fréquent, cf. robot aspirateur).

Propriétés du breadth-first search

Complet ?? Oui (si b est fini)

Optimal ?? Oui si la fonction de coût est une fonction non décroissante en la profondeur du noeud⁵ ; pas optimal en général

Temps ?? $1 + b + b^2 + b^3 + \dots + b^d = O(b^d)$, i.e., exponentiel en d

Espace ??

5. C'est le cas si tous les coûts sont constants (relativement fréquent, cf. robot aspirateur).

Propriétés du breadth-first search

Complet ?? Oui (si b est fini)

Optimal ?? Oui si la fonction de coût est une fonction non décroissante en la profondeur du noeud⁵ ; pas optimal en général

Temps ?? $1 + b + b^2 + b^3 + \dots + b^d = O(b^d)$, i.e., exponentiel en d

Espace ?? $O(b^d)$ (on a $O(b^{d-1})$ noeuds explorés et $O(b^d)$ noeuds dans la frontière)

L'espace est le principal problème : peut facilement générer 10^6 noeuds par sec.

5. C'est le cas si tous les coûts sont constants (relativement fréquent, cf. robot aspirateur).

Propriétés du breadth-first search

Temps et espace pour le BFS

Hypothèses : $b = 10$; 1 million de noeuds / sec et 1000 octets / noeuds

Profondeur	Noeuds	Temps	Espace
2	110	0.11 millisec	107 kilobytes
4	11110	11 millisec	10.6 megabytes
6	10^6	1.1 sec	1 gigabyte
8	10^8	2 min	103 gigabytes
10	10^{10}	3 heures	10 terabytes
12	10^{12}	13 jours	1 petabyte
14	10^{14}	3.5 ans	99 petabytes
16	10^{16}	350 ans	10 exabytes

Uniform-cost search

Quand tous les coûts sont égaux, le BFS est optimal car il explore d'abord les noeuds les moins profonds.

⇒ extension du BFS toujours optimale : **Uniform-cost search** où l'expansion choisit d'abord les noeuds avec le coût $g(n)$ minimum.

Différences avec le BFS :

- ordre de la file ($g(n)$ plutôt que FIFO)
- test objectif appliqué quand un noeud **est sélectionné pour l'expansion** (comme le GS générique, et plutôt que lors de la génération comme dans le BFS)
 - ▶ Pourquoi ?
- un test est ajouté pour vérifier s'il y a un meilleur chemin pour un noeud actuellement dans la frontière

Uniform-cost search

Quand tous les coûts sont égaux, le BFS est optimal car il explore d'abord les noeuds les moins profonds.

⇒ extension du BFS toujours optimale : **Uniform-cost search** où l'expansion choisit d'abord les noeuds avec le coût $g(n)$ minimum.

Différences avec le BFS :

- ordre de la file ($g(n)$ plutôt que FIFO)
- test objectif appliqué quand un noeud **est sélectionné pour l'expansion** (comme le GS générique, et plutôt que lors de la génération comme dans le BFS)
 - ▶ **Pourquoi ?** car le premier état objectif qui est généré peut être sur un chemin sous-optimal
- un test est ajouté pour vérifier s'il y a un meilleur chemin pour un noeud actuellement dans la frontière

Uniform-cost search

Expansion : d'abord les noeuds de coût minimum

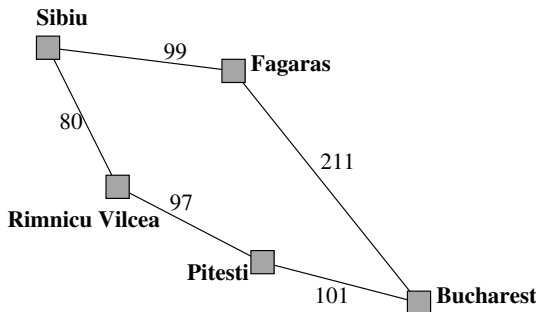
Implémentation :

frontier = priority queue ordonnée par PATH-COST (le plus petit d'abord)

⇒ ordre équivalent au BFS si tous les coûts sont égaux

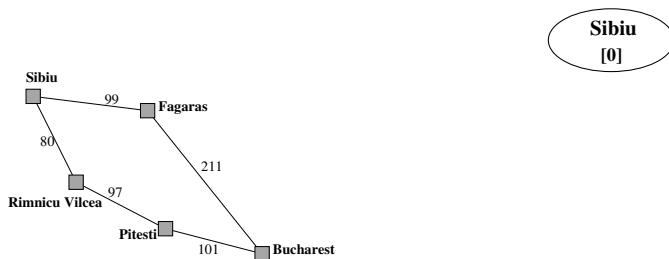
Uniform-cost search

Illustration : aller de Sibiu à Bucharest



Uniform-cost search

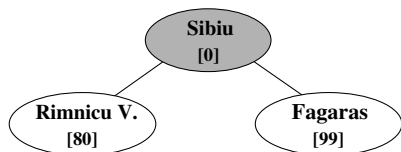
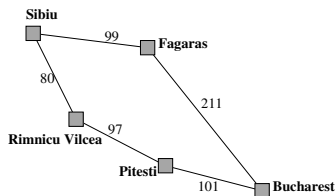
Illustration : aller de Sibiu à Bucharest



Priority queue : Sibiu [0]

Uniform-cost search

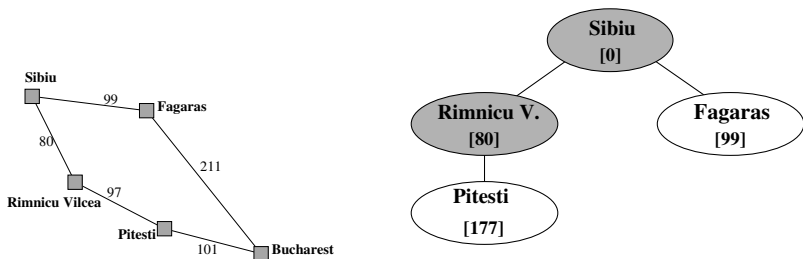
Illustration : aller de Sibiu à Bucharest



Priority queue : Rimnicu V. [80] → Fagaras [99]

Uniform-cost search

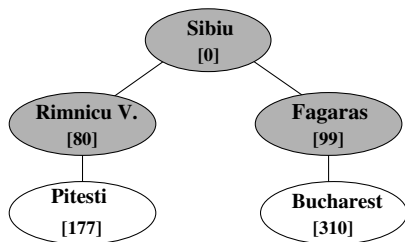
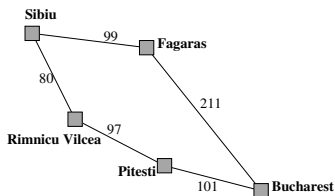
Illustration : aller de Sibiu à Bucharest



Priority queue : Fagaras [99] $\rightarrow$ Pitesti [177]

Uniform-cost search

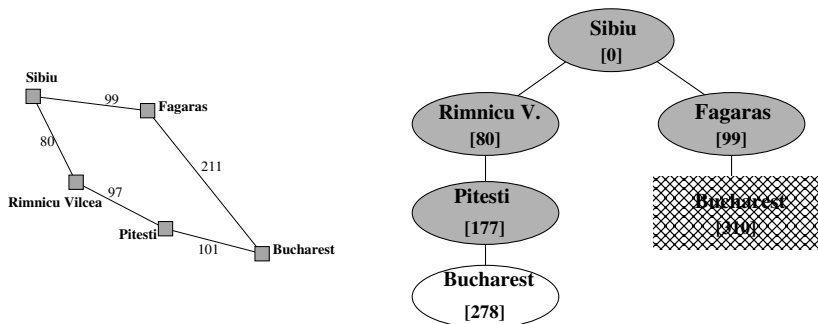
Illustration : aller de Sibiu à Bucharest



Priority queue : Pitesti [177] → Bucharest [310]

Uniform-cost search

Illustration : aller de Sibiu à Bucharest



Priority queue : Bucharest [278]

Uniform-cost search

```

function UNIFORM-COST-SEARCH(problem) returns sol or failure
  node ← MAKE-NODE(problem.INITIAL-STATE)
  node.PATH-COST ← 0
  frontier ← a priority queue ordered by PATH-COST,
    with node as the only element
  explored ← an empty set
  loop do
    if IS-EMPTY(frontier) then return failure
    node ← POP(frontier)
    if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
    add node.STATE to explored
    for each action in problem.ACTIONS(node.STATE) do
      child ← CHILD-NODE(problem, node, action)
      if child.STATE is not in explored or frontier then
        frontier ← INSERT(child, frontier)
      else if child.STATE is in frontier with higher PATH-COST then
        replace that frontier node with child

```


Propriétés de Uniform-cost search

Hypothèse : soit ε une (petite) constante > 0 telle que tout $c(s, a, s') \geq \varepsilon$.

Complet ?? Oui (grâce à l'hypothèse ci-dessus)

Optimal ?? Oui — les noeuds sont explorés en ordre croissant de $g(n)$

Temps ?? $O(b^{1+\lfloor C^*/\varepsilon \rfloor})$ où C^* est le coût de la solution optimale. Si tous les coûts sont égaux, $= O(b^{d+1})$ (= BFS avec niveau d'expansion supplémentaire)

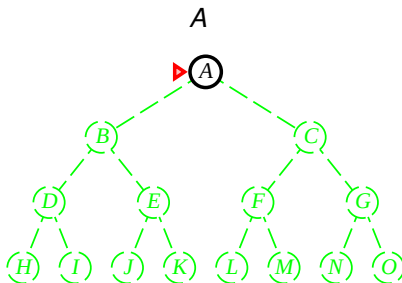
Espace ?? $O(b^{1+\lfloor C^*/\varepsilon \rfloor})$

Depth-first search

Expansion : d'abord les noeuds les plus profonds

Implémentation : GRAPH-SEARCH tel que

frontier = pile LIFO

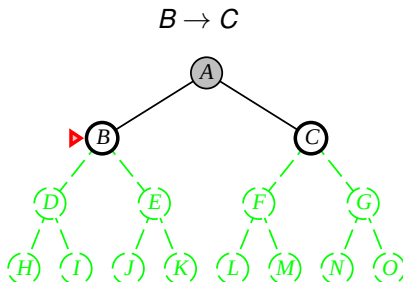


Depth-first search

Expansion : d'abord les noeuds les plus profonds

Implémentation : GRAPH-SEARCH tel que

frontier = pile LIFO



Remarque : C pourrait être au dessus de B dans la pile car ils ont la même profondeur (cela dépend des détails de l'implémentation et n'a pas d'importance). On illustre souvent le DFS comme ici, de gauche à droite.

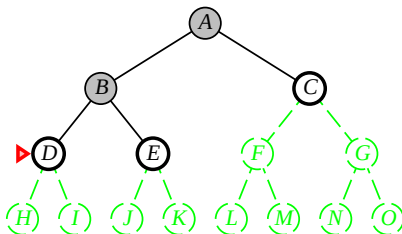
Depth-first search

Expansion : d'abord les noeuds les plus profonds

Implémentation : GRAPH-SEARCH tel que

frontier = pile LIFO

$D \rightarrow E \rightarrow C$



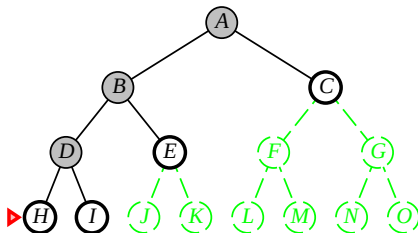
Depth-first search

Expansion : d'abord les noeuds les plus profonds

Implémentation : GRAPH-SEARCH tel que

frontier = pile LIFO

$H \rightarrow I \rightarrow E \rightarrow C$

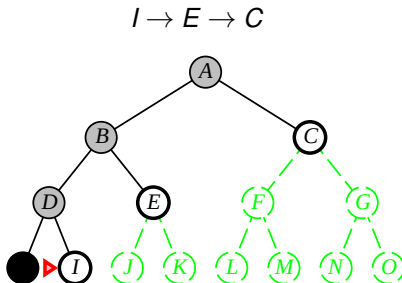


Depth-first search

Expansion : d'abord les noeuds les plus profonds

Implémentation : GRAPH-SEARCH tel que

frontier = pile LIFO



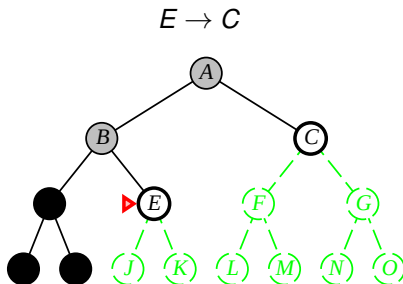
En noir : noeud dont tous les descendants ont été explorés : on peut le libérer en mémoire.

Depth-first search

Expansion : d'abord les noeuds les plus profonds

Implémentation : GRAPH-SEARCH tel que

frontier = pile LIFO



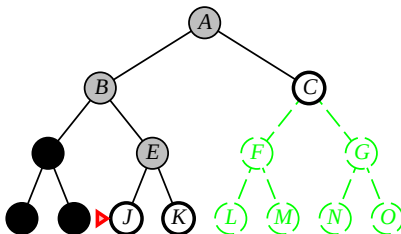
Depth-first search

Expansion : d'abord les noeuds les plus profonds

Implémentation : GRAPH-SEARCH tel que

frontier = pile LIFO

$J \rightarrow K \rightarrow C$

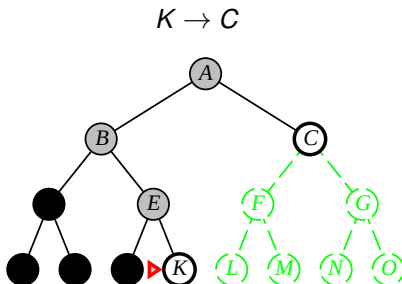


Depth-first search

Expansion : d'abord les noeuds les plus profonds

Implémentation : GRAPH-SEARCH tel que

frontier = pile LIFO

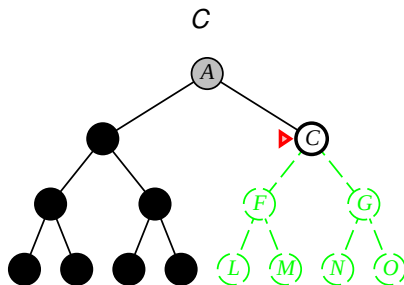


Depth-first search

Expansion : d'abord les noeuds les plus profonds

Implémentation : GRAPH-SEARCH tel que

frontier = pile LIFO



Remarque : facile à implémenter récursivement plutôt que via GS (voir cours SD1 et ci-après)

Propriétés du depth-first search

Les propriétés du DFS dépendent fortement de ce qu'on utilise :
TREE-SEARCH (TS) ou GRAPH-SEARCH (GS).

1^{ière} hypothèse : GS utilisé (on stocke les noeuds explorés)

Complet ??

Propriétés du depth-first search

Les propriétés du DFS dépendent fortement de ce qu'on utilise :
TREE-SEARCH (TS) ou GRAPH-SEARCH (GS).

1^{ière} hypothèse : GS utilisé (on stocke les noeuds explorés)

Complet ?? Oui si le nombre d'états dans l'espace d'états est fini
(chaque noeud est exploré au plus 1 fois)

Optimal ??

Propriétés du depth-first search

Les propriétés du DFS dépendent fortement de ce qu'on utilise :
TREE-SEARCH (TS) ou GRAPH-SEARCH (GS).

1^{ière} hypothèse : GS utilisé (on stocke les noeuds explorés)

Complet ?? Oui si le nombre d'états dans l'espace d'états est fini
(chaque noeud est exploré au plus 1 fois)

Optimal ?? Non, dès qu'un état objectif est atteint, il est retourné. Il
existe peut-être un autre état objectif à moindre coût non encore
exploré (à "droite" dans l'arbre)

Temps et espace ??

Propriétés du depth-first search

Les propriétés du DFS dépendent fortement de ce qu'on utilise :
TREE-SEARCH (TS) ou GRAPH-SEARCH (GS).

1^{ière} hypothèse : GS utilisé (on stocke les noeuds explorés)

Complet ?? Oui si le nombre d'états dans l'espace d'états est fini
(chaque noeud est exploré au plus 1 fois)

Optimal ?? Non, dès qu'un état objectif est atteint, il est retourné. Il
existe peut-être un autre état objectif à moindre coût non encore
exploré (à "droite" dans l'arbre)

Temps et espace ?? borné par le nombre d'états dans l'espace
d'états (qui peut être ∞)

Propriétés du depth-first search

2ième hypothèse : TS utilisé (pas de mémoire de ce qui est déjà exploré)

Complet ??

Propriétés du depth-first search

2ième hypothèse : TS utilisé (pas de mémoire de ce qui est déjà exploré)

Complet ?? Non (échoue si boucle)

Optimal ??

Propriétés du depth-first search

2ième hypothèse : TS utilisé (pas de mémoire de ce qui est déjà exploré)

Completeness ?? Non (échoue si boucle)

Optimal ?? Non, même si se termine ; pour les mêmes raisons que quand on utilise GS

Temps ??

Propriétés du depth-first search

2ième hypothèse : TS utilisé (pas de mémoire de ce qui est déjà exploré)

Completeness ?? Non (échoue si boucle)

Optimality ?? Non, même si se termine ; pour les mêmes raisons que quand on utilise GS

Time ?? $O(b^m)$ (peut générer tous les noeuds de l'arbre d'exploration, ce qui peut être beaucoup plus grand que le nombre d'états)

Space ??

Propriétés du depth-first search

2ième hypothèse : TS utilisé (pas de mémoire de ce qui est déjà exploré)

Complet ?? Non (échoue si boucle)

Optimal ?? Non, même si se termine ; pour les mêmes raisons que quand on utilise GS

Temps ?? $O(b^m)$ (peut générer tous les noeuds de l'arbre d'exploration, ce qui peut être beaucoup plus grand que le nombre d'états)

Espace ?? $O(bm)$: linéaire ! (taille de la frontière) : seul vrai avantage de la version TS, mais conséquent

Depth-limited search

Le DFS ne fonctionne pas dans des espaces infinis.

Extension : **Depth-limited search**, on limite la profondeur à ℓ (c-à-d, les noeuds de profondeur ℓ n'ont pas de successeurs)

- applicable dans un espace infini ;
- pas complet si $\ell < d$;
- peut ne pas être optimal si $\ell > d$;
- la limite peut être fixée en fonction de ce qu'on sait (par ex. le **diamètre**⁶ de la carte de la Roumanie est 9, donc $d \leq 9$ dans ce cas.)

6. Nombre d'étapes maximum entre n'importe quelle paire de villes.

Depth-limited search

Implémentation récursive :

```
function DEPTH-LIMITED-SEARCH(problem, limit) returns sol/fail/cutoff
  RECURSIVE-DLS(MAKE-NODE(problem.INITIAL-STATE), problem, limit)
```

```
function RECURSIVE-DLS(node, problem, limit) returns sol/fail/cutoff
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  else if limit = 0 then return cutoff
  else
    cutoff-occurred ?  $\leftarrow$  false
    for each action in problem.ACTIONS(node.STATE) do
      child  $\leftarrow$  CHILD-NODE(problem, node, action)
      result  $\leftarrow$  RECURSIVE-DLS(child, problem, limit-1)
      if result = cutoff then cutoff-occurred ?  $\leftarrow$  true
      else if result  $\neq$  failure then return result
    if cutoff-occurred ? then return cutoff else return failure
```

Iterative deepening search

Stratégie générale (mais utilisée ici avec le DFS) : itérer sur la limite.

```

function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution
  for depth  $\leftarrow$  0 to  $\infty$  do
    result  $\leftarrow$  DEPTH-LIMITED-SEARCH(problem, depth)
    if result  $\neq$  cutoff then return result
  end
  
```

Iterative deepening search

Limit = 0



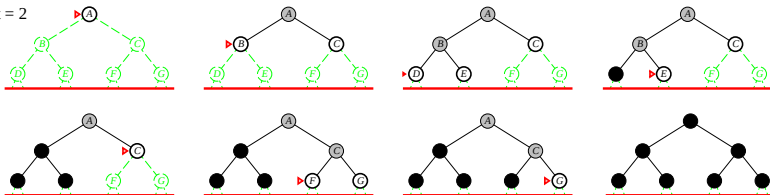
Iterative deepening search

Limit = 1



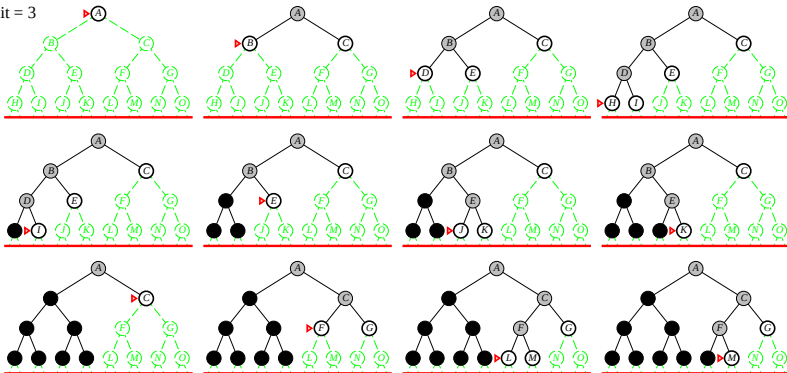
Iterative deepening search

Limit = 2



Iterative deepening search

Limit = 3



Propriétés de l'iterative deepening search

Parait contre-productif (des états sont générés plusieurs fois) **mais** combine les avantages du BFS et du DFS.

Complet ??

Propriétés de l'iterative deepening search

Parait contre-productif (des états sont générés plusieurs fois) **mais** combine les avantages du BFS et du DFS.

Comple ?? Oui si b est fini ($\simeq$ BFS)

Optimal ??

Propriétés de l'iterative deepening search

Parait contre-productif (des états sont générés plusieurs fois) **mais** combine les avantages du BFS et du DFS.

Comple ?? Oui si b est fini ($\simeq$ BFS)

Optimal ?? Oui si les coûts sont égaux ($\simeq$ BFS)

Temps ??

Propriétés de l'iterative deepening search

Parait contre-productif (des états sont générés plusieurs fois) **mais** combine les avantages du BFS et du DFS.

Complet ?? Oui si b est fini ($\simeq$ BFS)

Optimal ?? Oui si les coûts sont égaux ($\simeq$ BFS)

Temps ?? $db^1 + (d-1)b^2 + \dots + (1)b^d = O(b^d)$ ($\simeq$ BFS)

Espace ??

Propriétés de l'iterative deepening search

Parait contre-productif (des états sont générés plusieurs fois) **mais** combine les avantages du BFS et du DFS.

Complet ?? Oui si b est fini ($\simeq$ BFS)

Optimal ?? Oui si les coûts sont égaux ($\simeq$ BFS)

Temps ?? $db^1 + (d-1)b^2 + \dots + (1)b^d = O(b^d)$ ($\simeq$ BFS)

Space ?? $O(bd)$ (linéaire comme DFS version TS)

Complexité en temps similaire au BFS (surcoûts négligeables pour les calculs répétés). Ex. si $b = 10$, $d = 5$, et solution tout à fait à droite :

$$N(\text{IDS}) = 50 + 400 + 3,000 + 20,000 + 100,000 = 123,450$$

$$N(\text{BFS}) = 10 + 100 + 1,000 + 10,000 + 100,000 = 111,110$$

Comparaison des stratégies d'exploration non informées

Comparaison pour les versions TREE-SEARCH⁷.

Critère	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening
Complet ?	Oui*	Oui*	Non	Non*	Oui*
Temps	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$
Espace	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(bm)$	$O(b\ell)$	$O(bd)$
Optimal ?	Oui*	Oui	Non	Non	Oui*

* : voir conditions ci-avant (b est fini, etc.)

7. Pour GRAPH-SEARCH, les différences principales : DFS complet dans espaces finis et temps / espace bornés par le nombre d'états dans l'espace d'états.

Stratégies d'exploration informées (heuristiques)

Stratégies d'exploration informées

Une stratégie **informée** est une stratégie qui utilise une **connaissance spécifique au problème**, en plus de la seule définition du problème.

- Peut trouver des solutions de manière plus efficace que les stratégies non informées.
- Approche générale : TREE-SEARCH ou GRAPH-SEARCH en choisissant pour l'expansion le “meilleur” noeud d'abord (**best-first search**).
- La qualité d'un noeud est estimée via une **fonction d'évaluation**.

Best-first search

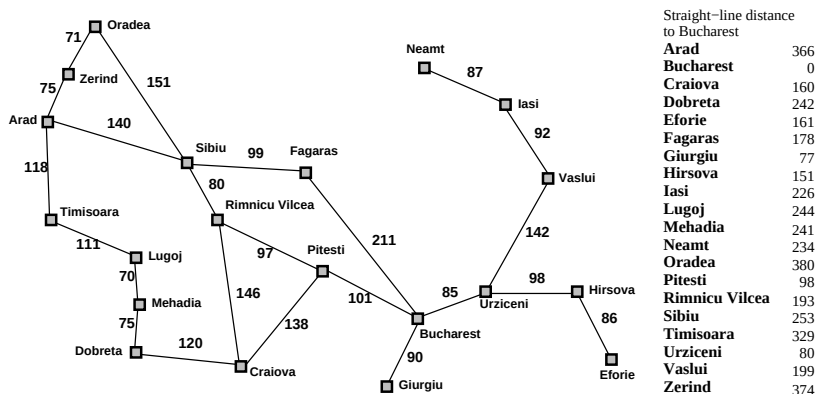
Idée : utilise une **fonction d'évaluation** $f(n)$ pour chaque noeud n

- $f(n) \simeq$ estimation de “ce noeud n est-il souhaitable / désirable?”, exprimé sous la forme d'un coût.
- Expansion : d'abord le noeud n qui a la plus petite valeur pour $f(n)$ (*frontier* est une file ordonnée par $f(n)$ décroissant)
- Le choix de f détermine la stratégie d'exploration.

Best-first search

- La plupart des algorithmes “best-first” utilisent une **fonction heuristique** $h(n)$ dans la définition de $f(n)$:
 $h(n)$ = estimation du coût minimum pour aller de l'état du noeud n à l'état objectif le plus proche.
- Exemples de stratégies (cas particuliers du best-first search) :
 - ▶ Uniform-cost search : $f(n) = g(n)$
 - ▶ exploration gloutonne (**greedy** best-first search) : $f(n) = h(n)$
 - ▶ exploration A^* : $f(n) = g(n) + h(n)$

Exemple : Roumanie (coûts en km)



A droite : distance à vol d'oiseau d'une ville à Bucharest.

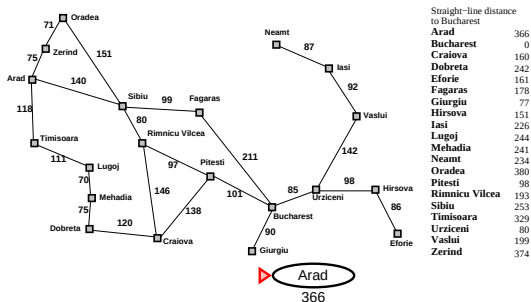
Greedy best-first search

Fonction d'évaluation $f(n) = h(n)$.

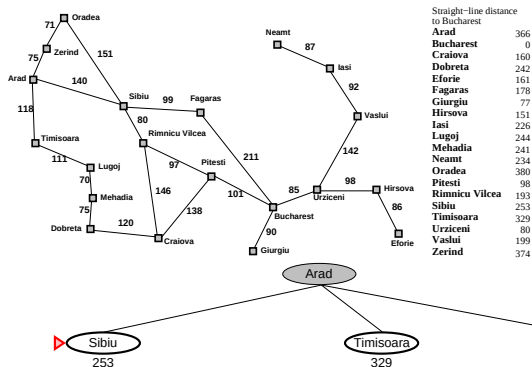
Exemple (Roumanie) : $h_{\text{SLD}}(n)$ = distance à vol d'oiseau (straight-line distance) de n à Bucarest.

L'exploration **gloutonne** (greedy) va sélectionner d'abord le noeud qui *semble* être le plus proche de l'objectif.

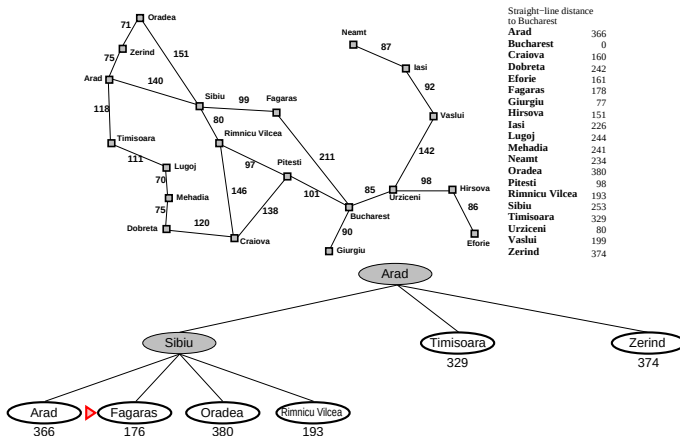
Greedy best-first search



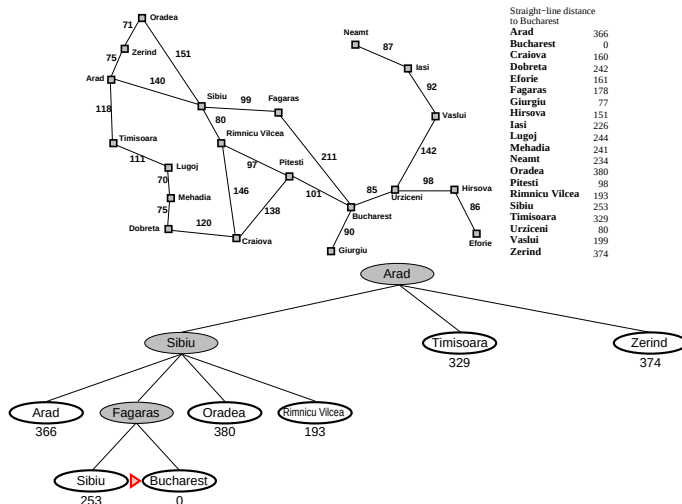
Greedy best-first search



Greedy best-first search



Greedy best-first search

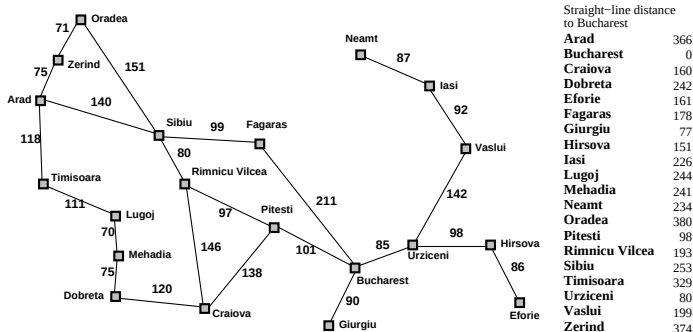


Propriétés du Greedy best-first search

Complet ??

Propriétés du Greedy best-first search

Complet ?? Non (boucles possibles).



Exemple : aller de Iasi à Fagaras : Iasi $\rightarrow$ Neamt $\rightarrow$ Iasi $\rightarrow$...

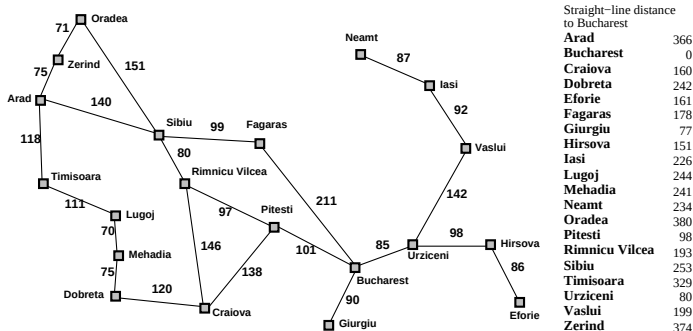
Mais complet si GRAPH-SEARCH dans espace fini

Propriétés du Greedy best-first search

Optimal ??

Propriétés du Greedy best-first search

Optimal ?? Non



Exemple : le chemin d'Arad à Bucharest via Sibiu et Fagaras (trouvé par Greedy) est 32 km plus long que le chemin via Sibiu, Rimnicu V. et Pitesti.

Propriétés du Greedy best-first search

Complexité en temps et en mémoire (version TREE-SEARCH) :

$$O(b^m),$$

mais une bonne fonction heuristique peut réduire la complexité de manière significative.

En pratique $\Rightarrow$ dépend du problème et de l'heuristique utilisée.

A* search

Fonction d'évaluation de l'exploration A* (oralement : “a-star”) :

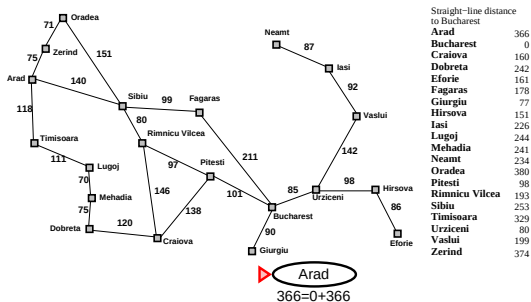
$$f(n) = g(n) + h(n)$$

- $g(n)$ = coût pour aller de l'état initial à n
- $h(n)$ = estimation du coût minimum pour aller de n à l'état objectif
- $f(n)$ = estimation du coût minimum total (de l'état initial à l'état objectif) en passant par n

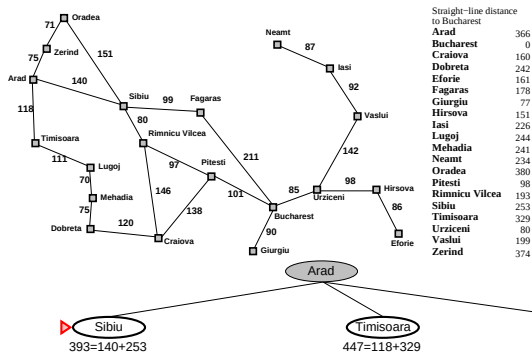
Algorithme identique à UNIFORM-COST-SEARCH sauf que l'on utilise $g + h$ au lieu de g pour les priorités.

⇒ A* est complet et optimal quand $h(n)$ respecte certaines conditions (voir ci-après)

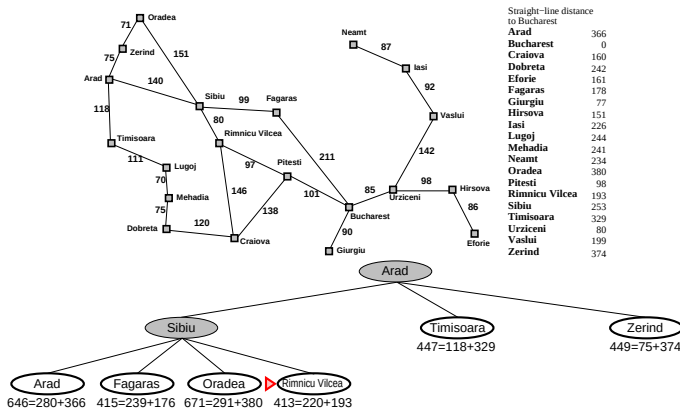
Exemple : A* search



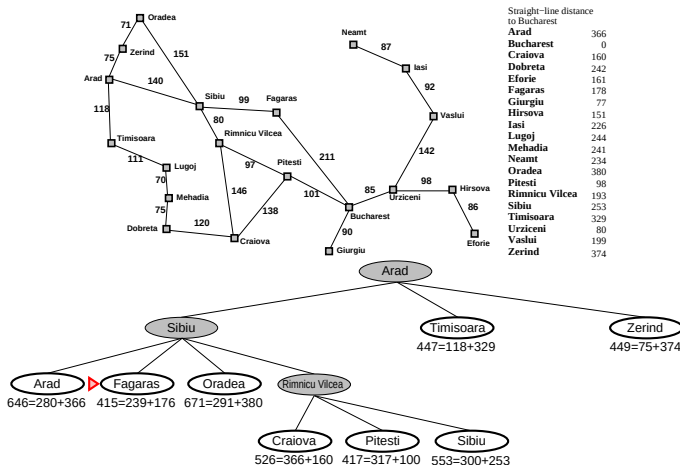
Exemple : A* search



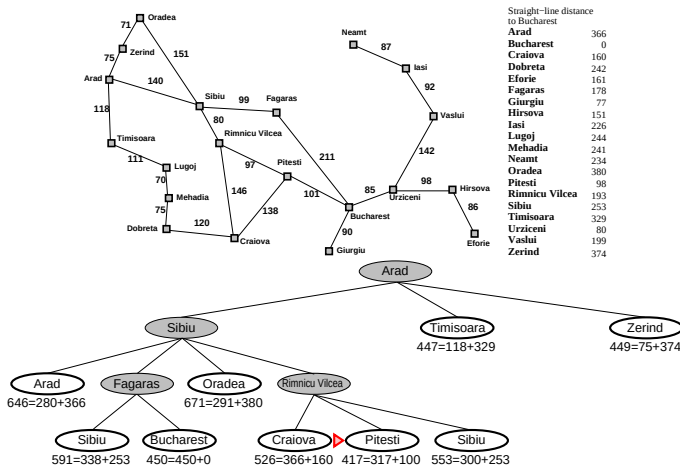
Exemple : A* search



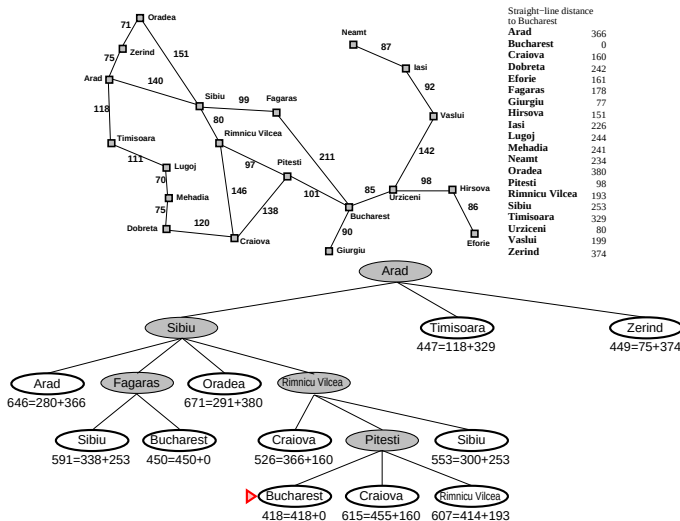
Exemple : A* search



Exemple : A* search



Exemple : A* search



Conditions d'optimalité

Définition

Une fonction heuristique $h(n)$ est dite **admissible** si elle ne surestime jamais le coût pour atteindre l'objectif, i.e.,

- $h(n) \leq h^*(n)$ où $h^*(n)$ est le coût réel (minimum) pour atteindre l'objectif depuis n ;
- $h(n) \geq 0$ et $h(G) = 0$ pour tout objectif G .

Exemple : $h_{\text{SLD}}(n)$ est admissible (la distance à vol d'oiseau ne surestime jamais la distance routière réelle).

Conditions d'optimalité

Définition

Une fonction heuristique $h(n)$ est dite **consistante** si

- $h(n) \geq 0$ et $h(G) = 0$ pour tout objectif G ; et
- $h(n) \leq c(n, a, n') + h(n')$;

pour tout noeud n et tout successeur n' de n .

- Condition plus forte que d'être simplement admissible (exercice).
- Forme de l'**inégalité triangulaire** (tout côté d'un triangle ne peut être plus long que la somme des longueurs des 2 autres côtés).

Conditions d'optimalité

Théorème

La version TREE-SEARCH d' A^ est optimale si $h(n)$ est admissible.*

La version GRAPH-SEARCH d' A^ est optimale si $h(n)$ est consistante.*

On prouve la deuxième assertion, qui est souvent plus utile en pratique.

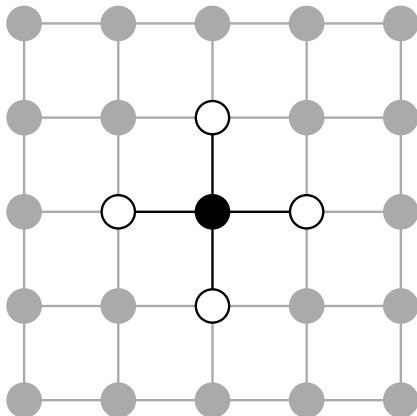
Propriété de séparation du graphe d'état de GS

Avec un GRAPH-SEARCH (GS), la frontière **sépare** le graphe d'états en une région explorée et une région inexplorée : tout chemin depuis l'état initial vers un état inexploré doit passer par un état de la frontière.

⇒ GS effectue une exploration systématique des états, un par un.

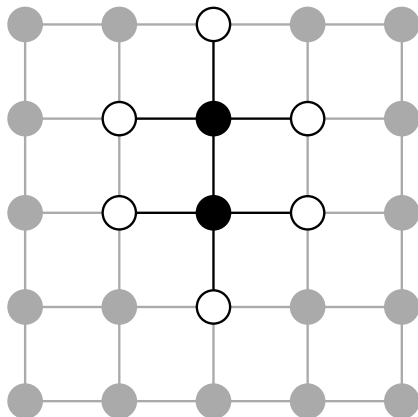
Propriété de séparation du graphe d'état de GS

Illustration sur une grille



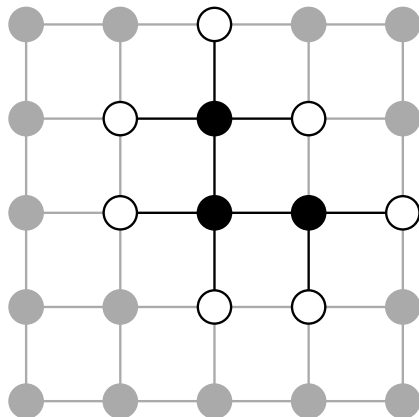
Propriété de séparation du graphe d'état de GS

Illustration sur une grille



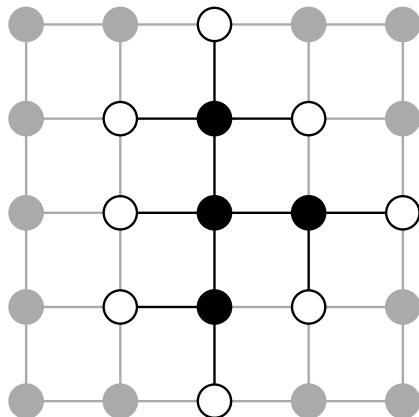
Propriété de séparation du graphe d'état de GS

Illustration sur une grille



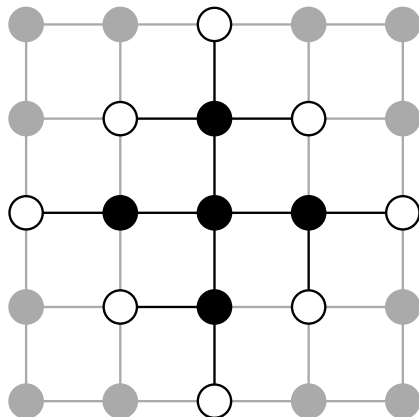
Propriété de séparation du graphe d'état de GS

Illustration sur une grille



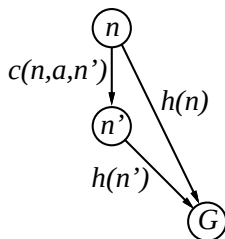
Propriété de séparation du graphe d'état de GS

Illustration sur une grille



Optimalité d'A* (preuve – GRAPH-SEARCH)

Etape 1 : si $h(n)$ est consistante, alors les valeurs de $f(n)$ le long de n'importe quel chemin (partant de la racine de l'arbre d'exploration) sont non décroissantes.



$$\begin{aligned}
 f(n') &= g(n') + h(n') \\
 &= g(n) + c(n, a, n') + h(n') \\
 &\geq g(n) + h(n) \\
 &= f(n)
 \end{aligned}$$

car $h(n) \leq c(n, a, n') + h(n')$ (h est consistante).
 Donc, $f(n') \geq f(n)$ et $f(n)$ est non décroissante sur n'importe quel chemin.

Optimalité d'A* (preuve – GRAPH-SEARCH)

Etape 2 : la séquence de noeuds choisie pour l'expansion par A* est non décroissante en $f(n)$.

Plaçons-nous en cours d'exploration et supposons qu'un noeud n ne soit pas le premier noeud à être choisi pour l'expansion. Alors, il existe un noeud de la frontière n' le long du chemin optimal allant du noeud initial à n (car tout chemin du noeud initial à n doit passer par un noeud de la frontière par la propriété de séparation de GS). Comme f est non décroissante (étape 1) le long de n'importe quel chemin, on a que $f(n') < f(n)$. Comme cette observation est vraie pour tout noeud n qui n'aurait pas été sélectionné pour l'expansion, on en conclut que le noeud choisi pour l'expansion est celui qui possède une valeur $f(n)$ minimum parmi tous les noeuds non explorés.

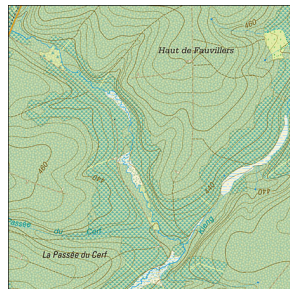
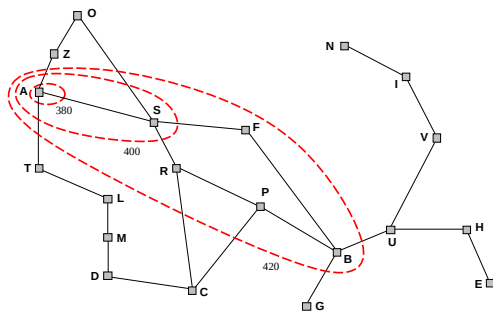
Optimalité d'A* (preuve – GRAPH-SEARCH)

Donc, la séquence de noeuds choisie pour l'expansion par A* est non décroissante en $f(n)$.

⇒ le premier objectif sélectionné pour l'expansion est optimal car $f(n) = g(n) + h(n)$ équivaut au coût réel pour les noeuds objectifs (car $h(G) = 0$) et tous les noeuds objectifs suivants seront au moins aussi coûteux. □

Contours

Comme f est non décroissante le long de tout chemin : elle définit des **contours** dans l'espace d'états.



Note : Avec l'uniform-cost search ($h(n) = 0$), les contours seraient "circulaires" autour de l'état initial. Plus h est précise, plus les contours se resserrent autour du chemin optimal.

Image de droite : Institut Géographique National de Belgique.

Propriétés d'A*

Soit C^* le coût de la solution optimale. Alors,

- A* explore tous les noeuds tels que $f(n) < C^*$.
- A* peut sélectionner certains noeuds tels que $f(n) = C^*$ (sur le “contour optimal”) avant de sélectionner un état objectif.
- A* ne sélectionne jamais un noeuds tel que $f(n) > C^*$.

Ce dernier point implique que certains sous-arbres de l'arbre d'exploration ne seront jamais visités ([pruning](#)).

Par exemple Timisoara (et son sous-arbre) ne sera jamais sélectionné sur la carte de la Roumaine, alors que c'est un fils de la racine.

Propriétés d'A*

Parmi les algorithmes optimaux qui explorent des chemins depuis l'état initial et qui utilisent la même information (heuristique) :

A* est d'une efficacité optimale

ce qui signifie qu'aucun autre algorithme de ce type n'explorera moins de noeuds que A* (sauf éventuellement pour briser des égalités parmi les noeuds tels que $f(n) = C^*$).

C'est dû au fait que les algorithmes qui n'explorent pas tous les noeuds tels que $f(n) < C^*$ courent le risque de manquer la solution optimale.

Propriétés d'A*

Complet ?? Oui si tous les “step costs” sont $\geq \epsilon$ et b est fini. Dans ce cas, il y a un nombre fini de noeuds avec un coût $\leq C^*$.

Optimal ?? Oui, si l'heuristique est admissible (TS) ou consistante (GS).

Temps et espace ?? Pour beaucoup de problèmes, le nombre d'états à l'intérieur du contour “objectif” est souvent exponentiel en la taille de la solution. De plus, on garde tous les noeuds en mémoire (comme dans tout GRAPH-SEARCH).

A* est complet, optimal et d'efficacité optimale (parmi les algorithmes similaires) **mais** reste impraticable pour de nombreux problèmes réels (principalement à cause de la mémoire).

Fonctions heuristiques

La qualité d'une stratégie d'exploration informée dépend de la fonction heuristique $h(n)$ choisie.

Exemple : pour le 8-puzzle.

$h_1(n)$ = nombre de pièces mal placées

$h_2(n)$ = somme des distances de **Manhattan** (i.e., # de carrés verticaux / horizontaux séparant chaque pièce de sa position finale)

7	2	4
5		6
8	3	1

Start State

1	2	3
4	5	6
7	8	

Goal State

$$h_1(S) = ??$$

$$h_2(S) = ??$$

Fonctions heuristiques

La qualité d'une stratégie d'exploration informée dépend de la fonction heuristique $h(n)$ choisie.

Exemple : pour le 8-puzzle.

$h_1(n)$ = nombre de pièces mal placées

$h_2(n)$ = somme des distances de **Manhattan** (i.e., # de carrés verticaux / horizontaux séparant chaque pièce de sa position finale)

7	2	4
5		6
8	3	1

Start State

1	2	3
4	5	6
7	8	

Goal State

$$h_1(S) = ?? \ 6$$

$$h_2(S) = ??$$

Fonctions heuristiques

La qualité d'une stratégie d'exploration informée dépend de la fonction heuristique $h(n)$ choisie.

Exemple : pour le 8-puzzle.

$h_1(n)$ = nombre de pièces mal placées

$h_2(n)$ = somme des distances de **Manhattan** (i.e., # de carrés verticaux / horizontaux séparant chaque pièce de sa position finale)

7	2	4
5		6
8	3	1

Start State

1	2	3
4	5	6
7	8	

Goal State

$$h_1(S) = ?? \ 6$$

$$h_2(S) = ?? \ 4+0+3+3+1+0+2+1 = 14$$

Dominance

Bien sûr $h_1(n)$ et $h_2(n)$ ne surestiment jamais le coût réel de la solution.

Si $h_2(n) \geq h_1(n)$ pour tout n (toutes 2 consistantes), alors h_2 domine h_1 et est (en général) une meilleure stratégie.

Coûts typiques :

$d = 14$ IDS = 3,473,941 noeuds

$A^*(h_1) = 539$ noeuds

$A^*(h_2) = 113$ noeuds

$d = 24$ IDS $\approx$ 54,000,000,000 nodes

$A^*(h_1) = 39,135$ noeuds

$A^*(h_2) = 1,641$ noeuds

Dominance

Soit deux fonctions heuristiques consistantes h_a , h_b . Alors,

$$h(n) = \max(h_a(n), h_b(n))$$

est également consistante et domine h_a et h_b .

⇒ en général meilleur d'utiliser une fonction heuristique avec de grandes valeurs, pour peu qu'elle soit consistante et que son temps de calcul ne soit pas trop long.

Relaxation de problèmes

Des heuristiques admissibles peuvent être dérivées du coût de la solution **exacte** d'une version **relaxée** du problème.

Définition

Un problème pour lequel il y a moins de restrictions sur les actions est un problème **relaxé**.

Par exemple, si on simplifie les règles du 8-puzzle :

- une pièce peut être déplacée *n'importe où* $\rightarrow h_1(n)$ donne la solution optimale ;
- une pièce peut être déplacée *sur tout carré adjacent* $\rightarrow h_2(n)$ donne la solution optimale.

Idée : la solution optimale d'un problème relaxé est égale ou meilleure à la solution optimale du problème original, et donc l'heuristique est admissible par construction.

Résumé du chapitre

Résumé du chapitre

Ce chapitre a introduit les méthodes (explorations) qu'un agent utilise pour sélectionner des actions dans un environnement déterministe, observable, statique et entièrement connu.

- Formulation du problème : requiert d'abstraire les détails réels pour définir un espace d'états pouvant être exploré.
- Il existe diverses stratégies d'exploration non informées.
- **Iterative deepening search** n'utilise qu'un espace mémoire de complexité linéaire et pas beaucoup plus de temps que d'autres algorithmes non informés.
- GRAPH-SEARCH peut être exponentiellement plus efficace que TREE-SEARCH.

Résumé du chapitre

- Une **fonction heuristique** estime le coût des plus courts chemins.
- Une bonne fonction heuristique peut réduire de façon considérable le coût d'une exploration
- **Greedy best-first search** sélectionne d'abord le noeud ayant la plus petite valeur de h — incomplet et pas toujours optimal
- A^* sélectionne d'abord le noeud ayant la plus petite valeur de $g + h$ — complet, optimal et d'efficacité optimale (pour des algorithmes du même type).
- Une heuristique **admissible** peut être dérivée de la solution exacte de problèmes relaxés.